

6 测试说明

6.1 仿真测试

项目已经提供了仿真文件sim/mycpu.tb和测试文件tests/coe.

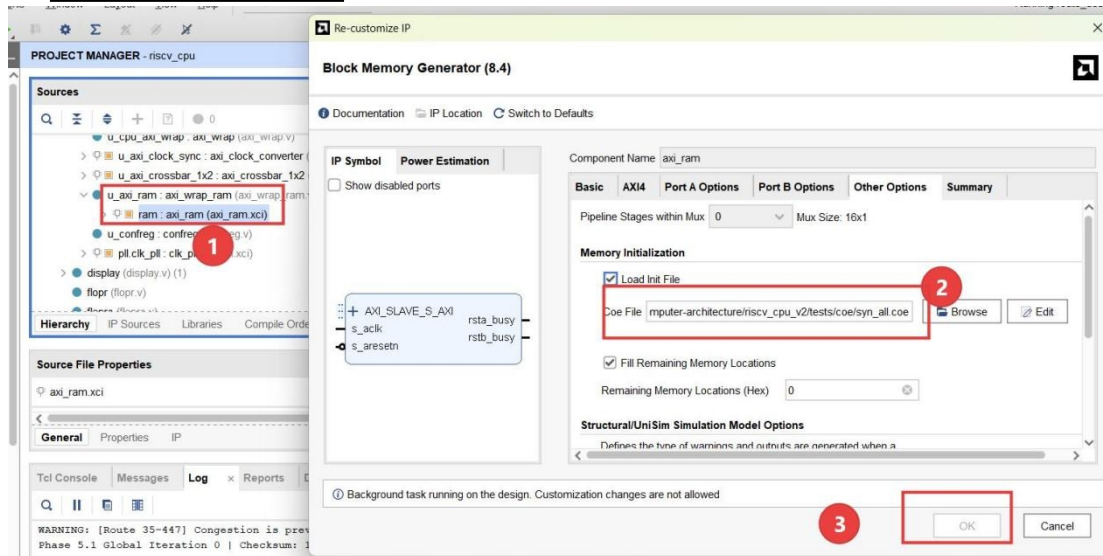
1. 首先修改仿真文件顶部的TEST_CASE_NAME:

```
//[basic_37, priv_exceptA, priv_exceptB, mul, mul_div_rem, syn_all]
`define TEST_CASE_NAME "basic_37"
`define TRACE_DIR "../.../tests/traces/"
```

2. 打开soc_axi_lite_top.v 中的`define SIMULATION宏定义

```
64 `define SIMU_USE_PLL 0 //set 0 to speed up simulation
65 `define SIMULATION
```

3. 修改ram中的coe文件:



各个阶段所需要的测试用例如下表:

任务描述	TEST_CASE_NAME	仿真速度
37条基础指令	basic_37	中等
动态分支预测器	basic_37	中等
特权与异常处理指令	priv_exceptA、priv_exceptB	快
扩展指令	mul、mul_div_rem	快
扩展内容	syn_all	较慢

(1) 请先通过basic_37测试用例再完成动态分支预测器。目前仿真文件没有死循环检测逻辑，仿真时请观察控制台打印日志或波形图中的 debug_wb_pc、

debug_wb_rf_wdata等信号是否发生变化，若长时间未变化或控制台未任何输出，立即终止仿真，排查错误原因。

(2) priv_exceptA、priv_exceptB测试用例均需分别测试。

(3) mul只包含乘法指令；mul_div_rem包含乘法、除法和取余指令。

(4) syn_all属于综合测试，在完成扩展内容后涉及的功能仿真与性能分析以该测试为准。

(5) 未经允许禁止修改仿真文件、soc_axi_lite_top.v文件、CEO文件、trace文件，一经发现则课程成绩不合格。

其他说明：

(1) PC复位时应该置为：32'h80000000

(2) debug_wb_rf_wen在已有基础上建议加上RdW != 5'b0逻辑。

6.2 综合流程

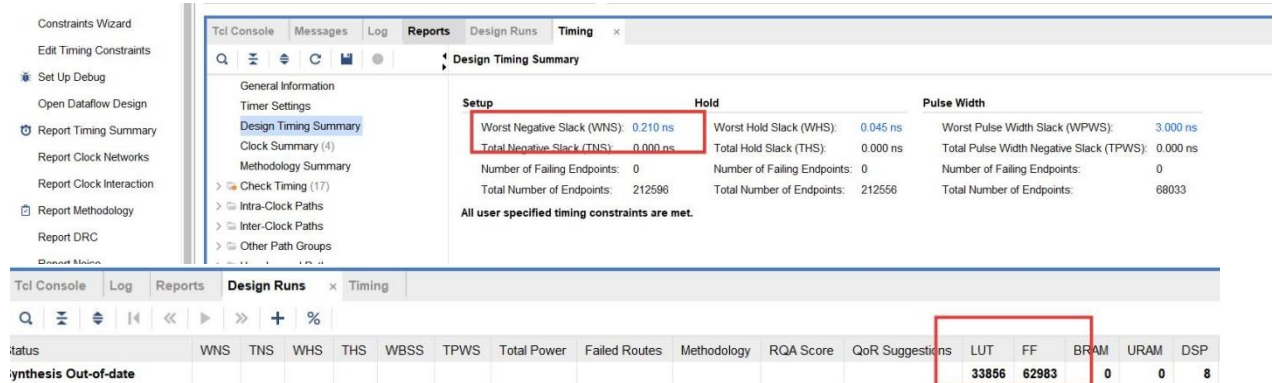
运行综合前确保项目的syn/soc_lite.xdc已经导入。

同时将soc_axi_lite_top.v 顶部的SIMULATION宏注释掉。

```
`define SIMU_USE_PLL 0 //set 0 to speed up simulation
// `define SIMULATION
```

点击“Run Synthesis”，运行综合，分析工具产生的警告信息（Warning和Critical Warning），判断是否需要解决。若不需要解决，在报告中说明不需要解决的原因。

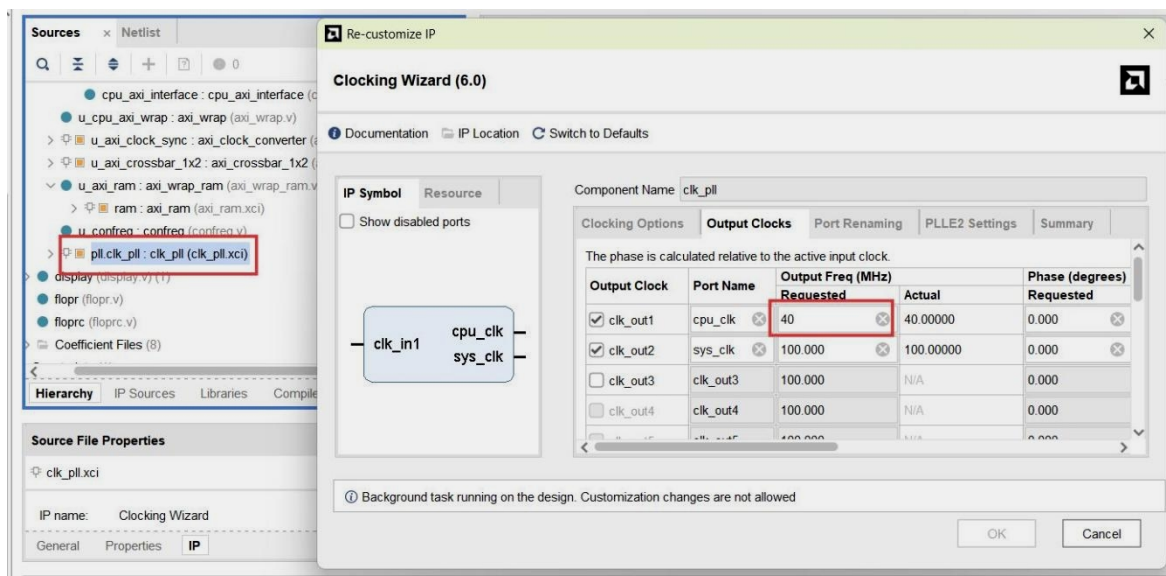
综合成功后，需要在报告中呈现时序信息和资源使用情况，参考如下：



(1) 若未在报告中呈现警告分析内容，则扩展内容最高得5分；

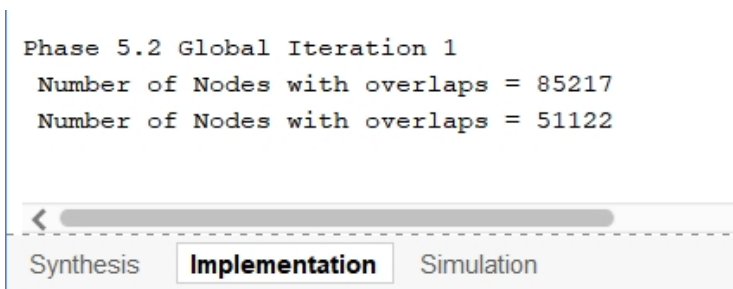
(2) 若未呈现时序信息，则扩展内容最高得5分；

(3) 若需修改CPU主频则参考（若低于40MHz则无任何额外加分）。



6.3 上板流程

在通过综合后，可选择继续完成Run Implementation（布局布线）。如果设计中寄存器使用过多或模块连接复杂密集，则该过程会消耗很长时间（几小时到几十小时不等，电脑性能越好时间越短），甚至可能直接失败。如下图所示，可以看到overlaps特别大（说明电路拥塞非常严重），工具很难优化到0，即使优化成功时序也有可能不通过。这种情况建议优化电路设计后（比如减少寄存器堆的使用，降低扇入扇出等）再进行布局布线。



在完成布局布线后，需确保WNS为正才能进行Generate Bitstream，然后连接至FPGA开发板进行上板测试。

若上板，报告中需提供布局布线后的时序信息和FPGA数码管显示数值。

7 调试说明

在过往的四次计组实验中，我们提供的测试程序都很简单，其检测机制都是通过运行指令后检测某个寄存器的值是否与预期一致。同学们可以通过观察波形图，然后跟着源程序执行顺序进行比对，从而定位错误。此次综合设计的测试程序包含上百条指令，且程序运行次数高达几万次，再采用逐条对比的方式就不太现实，于是就产生了新的对比机制。

7.1 仿真对比机制

以下所有文件均在tests文件夹下：

- (1) coe: 导入RAM的指令，需要根据不同任务导入不同的文件；
- (2) assembly: 所有测试用例的汇编程序；
- (3) traces.txt: 仿真时的比对文件。

trace.txt每行数据格式如下：

1 80000038 02 00000080

第一位1为写寄存器标志（对应debug_wb_rf_wen），80000038为当前执行PC，02为写寄存器号（对应debug_wb_rf_wnum），00000080为写入寄存器的数据（对应debug_wb_rf_wdata）。

trace.txt文件的内容是由Spike模拟器（一个较为完整的RISCV模拟器）运行测试程序后的输出结果，仿真对比的就是自己设计的CPU的输出和该trace.txt文件是否一致。当我们设计的CPU或Cache实现错误时，在终端就会输出报错信息，如图：一开始PC就不一致，这种情况大概率就是流水线暂停逻辑没有处理好。

```
Test begin!
-----
[ 4683 ns] Error!!!
reference: PC = 0x80000038, wb_rf_wnum = 0x02, wb_rf_wdata = 0x00000080
mycpu      : PC = 0x8000003c, wb_rf_wnum = 0x02, wb_rf_wdata = 0x00080000
```

还有一种常见的情况就是PC相同，但wb_rf_wnum或wb_rf_wdata不一致，这种情况就可以结合波形图定位到出错的PC，然后在汇编文件中找到对应PC的指令，再结合程序上下文以及波形图中的关键信号（比如暂停信号、ALU输出、控制器信号等等）分析原因。

7.2 调试小技巧

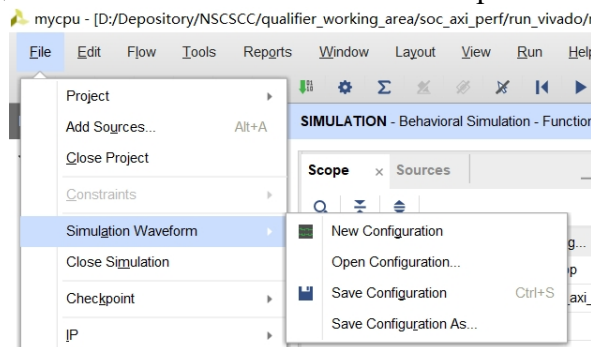
在仿真调试过程中，我们需要从波形图获取有效信息。为了避免重复拉取波形图信号，我们可以将已经配置好的波形图保存下来。在工程的后续调试中或者小组合作

时，可以导入已经配置好的波形图文件（当然，需要在同一个工程项目中）。以下是一个参考示例：

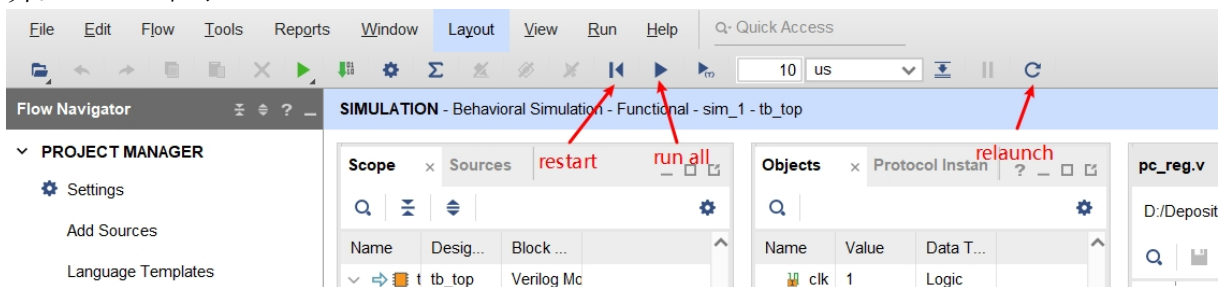
1. 导出配置好的波形图文件。快捷键ctrl+s，或者点击file-simulation waveform-save configuration as



2. 导入波形图文件。点击file-simulation waveform-open configuration。



3. 当我们导入好波形图后，我们需要restart以加载导入的波形图，然后点击run all命令，开始仿真。注意区分relaunch与restart用法。当修改设计代码、测试代码和设置ip核参数后，只用relaunch仿真才能生效。对于加载波形图跟修改波形图，只用restart并且run all即可。



同时，我们提供一些波形图配置的小技巧：

1. 信号量分组。多选住信号量后，右键可进行分组。对于有很多变量的分组，可以按住shift键，然后鼠标左键确定起点和终点，即可多选。
2. 善用搜索功能，选中一条波形，可通过数值索引到对应的位置。
3. 当信号量过多时，可能发生左侧信号量跟波形错位的情况，同学们应注意。
4. 当信号量过多时，可通过配置波形颜色，减轻用眼负担。

7.3 问题反馈

若发现问题，可第一时间反馈给助教。